
UNIT 6 HIGHER NORMAL FORMS

Structure

- 6.0 Introduction
- 6.1 Objectives
- 6.2 Multivalued Dependency
- 6.3 Fourth Normal Form (4NF)
- 6.4 Join Dependency
- 6.5 5NF
- 6.6 Other Normal Forms
- 6.7 Summary
- 6.8 Solutions/Answers

6.0 INTRODUCTION

In the previous unit of this block, you have gone through the concept of single-valued dependency, called functional dependency (FD). Further, the previous Unit discussed different forms of normalisations that can be arrived at by removing different types of single-valued dependency, viz. 1NF, 2NF, 3NF and BCNF. However, relational databases may have several other types of dependencies, which results in redundancy in a relation. Such dependencies, thus causes the update anomaly, insertion anomaly and deletion anomaly in a relation. Therefore, more normal forms have been designed to address this problem.

This Unit focuses on the higher normal forms, namely fourth normal form (4NF), which is based on the concept of multivalued dependency (MVD); and fifth normal form (5NF), which is based on the concept of Join dependency. The unit also introduces you to other normal forms. For more details on other normal forms, you may refer to the further readings. In general, these higher normal forms are not very popular among industries, however they propose good theoretical perspectives for the database design.

6.1 OBJECTIVES

After going through this unit, you should be able to:

- Explain the concept of multivalued dependencies;
- Perform normalisation up to 4NF;
- Explain the join dependency;
- Explain the 5NF;
- Define other normal forms.

6.2 MULTIVALUED DEPENDENCIES

An attribute in a relational model represents only a single value information. How will you represent the information if a particular attribute is multivalued? Such multivalued attributes would require that

information of all other attributes is repeated in an instance of the relation. This will result in multiple tuples in a single relation to represent information about one entity. The primary key of such a relation would be a composite key involving the primary key of the entity and the multivalued attribute. This situation becomes worse, if an entity has more than one multivalued attribute. Such an entity will be represented by several tuples. The following example explains this situation:

Example 1: Let us consider a relation ‘employee’.

emp (e#, project, tool)

An employee can work on several projects and an employee has skills in several tools, therefore, there is no functional dependency in this relation. However, this relation has two multivalued attributes: *project* and *tool*.

The attributes *project* and *tool* are assumed to be independent of each other, as any project can use any tool. The following table (not relation) defines this situation:

e# (Employee Number)	Project	Tool
E001	DBMS, Ecommerce	Python, Virtualization
E002	Ecommerce, Bank Automation	PostgreSQL, Virtualization
E003	Bank Automation	PostgreSQL

Figure 6.1: Sample Data

However, to represent this information through 1NF, you need to create the following relation:

<u>e#</u>	<u>project</u>	<u>tool</u>
E001	DBMS	Python
E001	DBMS	Virtualization
E001	Ecommerce	Python
E001	Ecommerce	Virtualization
E002	Ecommerce	PostgreSQL
E002	Ecommerce	Virtualization
E002	Bank Automation	PostgreSQL
E002	Bank Automation	Virtualization
E003	Bank Automation	PostgreSQL

Figure 6.2: 1NF relation of data of Figure 6.1

This relation has no FD and primary key of the relation is composite key (*e#, project, tool*), therefore, the relation is in 3NF and BCNF. However, it suffers from the problem of redundancy, for example, the employee E001 is working on DBMS is appearing twice, so is that E001 knows the tool Python. This may lead to update anomaly. Further, you cannot insert information about a new employee, who knows the tools PostgreSQL, but has not been assigned to any project. In addition, if you remove the employee E003 from the Bank Automation project, the information that E003 has skill in PostgreSQL would be lost. Thus, the relation suffers from update, insertion and deletion anomalies.

How can you now normalise the relation, as this contains no FD? For addressing the issues related to such relations, we may first define the concept of multivalued dependency, which relates to relations that contain more than one multivalued attribute. Let us define the multivalued dependency formally for a relation $R(X, Y, Z)$, where X, Y and Z are a set of attributes.

Multivalued dependency (MVD): Given a relation $R(X, Y, Z)$, a MVD $X \twoheadrightarrow Y$ will hold in relation R if for a specific value of attribute set X , there is an associated set of values of attribute set Y , such that these

multiple-associated (zero or more) values of attributes Y depends only on the value of attribute X . Further, values of attribute set Y have no dependence on the value of attribute set Z .

Hence, if $MVD X \twoheadrightarrow Y$ holds in R , another $MVD X \twoheadrightarrow Z$ would also hold, as the function of the attribute set Y and attribute set Z is symmetrical.

In the *Example 1* given above, the employee number can determine all the projects, employee is working on, and also an employee number can determine all the tools in which that employee is skillful. Further, both the *project* and *tool* attributes are independent of each other. Therefore, the following MVDs hold in relation of example 1:

$e\# \twoheadrightarrow project$
 $e\# \twoheadrightarrow tool$

Let us now define the concept of MVD in a different way. Consider the relation $R(X, Y, Z)$ having a multi-valued set of attributes Y associated with a value of X . Assume that the attributes Y and Z are independent, and Z is also multi-valued. Now, more formally, $X \twoheadrightarrow Y$ is said to hold for $R(X, Y, Z)$ if $t1$ and $t2$ are two tuples in R that have the same values for attributes X ($t1[X] = t2[X]$) then R also contains tuples $t3$ and $t4$ (not necessarily distinct) such that:

$t1[X] = t2[X] = t3[X] = t4[X]$
 $t3[Y] = t1[Y]$ and $t3[Z] = t2[Z]$
 $t4[Y] = t2[Y]$ and $t4[Z] = t1[Z]$

For example, consider Figure 6.2, the two such tuples are:

Tuple 1: E001	DBMS	Python
Tuple 4: E001	Ecommerce	Virtualization

Then two more tuples must exist as follows:

E001	DBMS	Virtualization
E001	Ecommerce	Python

Please check these two are Tuple 2 and Tuple 3 in Figure 6.2

We are, therefore, requiring that every value of Y appears with every value of Z to keep the relation instances consistent. In other words, the above conditions insist that Y and Z are determined by X alone and there is no relationship between Y and Z , since Y and Z appear in every possible pair and hence these pairings present no information and are of no significance. Only if some of these pairings were not present, there would be some significance in the pairings.

(Note: If Z is single-valued and functionally dependent on X then $Z1 = Z2$. If Z is multivalued dependent on X then $Z1 \neq Z2$).

The theory of multivalued dependencies is very similar to that for functional dependencies. Given a set of MVDs, say D , you can find D^+ , the closure of D using a set of axioms. We do not discuss the axioms here. You may refer to this topic in further readings.

Before explaining the 4NF of a relation, one more term needs to be defined. It is the Trivial MVD, which is defined next.

Trivial MVD: A MVC $X \twoheadrightarrow Y$ is called trivial MVD if either Y is a subset of X or X and Y together form the relation R .

The MVD is trivial since it results in no constraints being placed on the relation. For example, consider a relation *dependent* (*e#*, *edependent#*). An employee can have zero or more dependents, therefore, *e#* uniquely determines the **values** of *edependent#*. However, this MVD is trivial, as *e#*, *edependent#* together forms the relation *dependent*. This *dependent* relation cannot be decomposed any further.

Therefore, a relation having non-trivial MVDs must have at least three attributes; two of them would be multivalued and not dependent on each other. Non-trivial MVDs result in the relation having some constraints on it since all possible combinations of the multivalued attributes are then required to be present in the relation. In the next section, we discuss how MVD can be used to decompose a relation into fourth normal form.

6.3 FOURTH NORMAL FORM

In this section, first we define the fourth normal form (4NF) and then present an example about how MVD can be used to decompose a relation to 4NF.

A relation R is in 4NF if for all the multivalued dependencies $(X \twoheadrightarrow Y)$ any one of the following clauses holds:

- the multivalued dependencies $(X \twoheadrightarrow Y)$ is trivial,
- X is a candidate key for R .

The dependency $X \twoheadrightarrow \emptyset$ or $X \twoheadrightarrow Y$ in a relation $R(X, Y)$ is trivial, since they must hold for all $R(X, Y)$. In this case, $R(X, Y)$ is in 4NF. Similarly, if in a relations $R(A, B, C)$ with only three attributes, if a trivial MVD $(A, B) \twoheadrightarrow C$ holds, then $R(A, B, C)$ is in 4NF.

If a relation has more than one multivalued attribute, you should decompose it into the fourth normal form using the following rules of decomposition:

For a relation $R(X, Y, Z)$, if it contains two nontrivial MVDs $X \twoheadrightarrow Y$ and $X \twoheadrightarrow Z$, then decompose the relation into $R_1(X, Y)$ and $R_2(X, Z)$ or more specifically, if there holds a non-trivial MVD in a relation $R(X, Y, Z)$ of the form $X \twoheadrightarrow Y$, such that $X \cap Y = \emptyset$, that is the set of attributes X and Y are disjoint, then R must be decomposed to $R_1(X, Y)$ and $R_2(X, Z)$, where Z represents all attributes other than those in X and Y .

Intuitively R is in 4NF if

- (1) All dependencies are a result of keys.
- (2) When multivalued dependencies exist, a relation should not contain two or more independent multivalued attributes.

The decomposition of a relation to achieve 4NF would normally result in not only reduction of redundancies but also avoidance of anomalies.

Example 2: Normalise the relation *emp* (*e#*, *project*, *tool*) shown in Figure 6.2 using the MVDs:

$e# \twoheadrightarrow \text{project}$ and $e# \twoheadrightarrow \text{tool}$.

The relation has no FD, but two MVDs, therefore, the relation is in BCNF but not 4NF. To decompose the relation into 4NF, you may use any of the MVD. The decomposed relation would be:

empproj (*e#*, *project*) with MVD $e# \twoheadrightarrow \text{project}$, which is now a trivial MVD; and

emptool (*e#*, *tool*) with MVD $e# \twoheadrightarrow tool$, which is a trivial MVD.

On decomposition of the relation in Figure 6.2, by taking the projections as stated above, the relational state of the decomposed relations would be:

<i>empproj</i>		<i>emptool</i>	
<u><i>e#</i></u>	<u><i>project</i></u>	<u><i>e#</i></u>	<u><i>tool</i></u>
E001	DBMS	E001	Python
E001	Ecommerce	E001	Virtualization
E002	Ecommerce	E002	PostgreSQL
E002	Bank Automation	E002	Virtualization
E003	Bank Automation	E003	PostgreSQL

Figure 6.3: Decomposition of *emp* (*e#*, *project*, *tool*) relation to 4NF

You can observe the following in Figure 6.3

- The key to relations *empproj* and *emptool* are (*e#*, *project*) and (*e#*, *tool*) respectively.
- There is no redundancy in *empproj* and *emptool* relations.
- Both the relations do not suffer from any anomaly.

So far, you are able to decompose a relation based on FD and MVD. Are there any other forms of dependencies? Researcher have shown several kinds of dependencies. However, for this course we will discuss about one more type of dependency, called the join dependency, which is discussed next.

☞ Check Your Progress 1

- 1) What are Multi-valued Dependencies? When can you say that a constraint X is multi-valued dependency?
.....
.....
.....
- 2) Identify the MVDs in the following relation. Decompose the relation into 4NF using the MVDs.

EMP

ENAME	PNAME	DNAME
Rohan	Big Data	AI Unit
Rohan	Machine Learning	Analytics
Rohan	Big Data	Analytics
Rohan	Machine Learning	AI Unit

- 3) Convert the following relation to 4NF relations.

SUPPLY

SNAME	PARTNAME	PROJNAME
XYZ Pvt Ltd	Bolt	Big Data
XYZ Pvt Ltd	Nut	Machine Learning
ABC Ltd	Bolt	Machine Learning
Info Comm Ltd	Nut	AI
ABC Ltd	Nail	Big Data
ABC Ltd	Bolt	Big Data
XYZ Pvt Ltd	Bolt	Machine Learning

.....

.....

.....

6.4 JOIN DEPENDENCIES

As discussed in the previous section, a relation that suffers from insertion, update and deletion anomalies is decomposed using either FD or MVD. The normal forms require that a given relation R , if not in the given normal form, should be decomposed in two relations to meet the requirements of the normal form. However, in some cases, a relation can have problems like redundancy leading to anomalies, yet it cannot be decomposed in two relations without loss of information. In such cases, it may be possible to decompose the relation in three or more relations. When does such a situation arise? Such cases normally happen when a relation has at least three attributes such that all those values are totally independent of each other. It may also be the case when a ternary relationship exists among three entities.

Following example explains this in detail. Consider a relation:

ProjToolEmp (projectid, toolid, empid)

There are no constraints on this relation that is:

- Any project can use any tools
- Any tool can be used by any employee
- Any employee can work on any project
- No employee would use all the tools
- No employee would work on all the projects
- No project uses all the tools
- All three attributes are independent of each other

Assume that the relation has the following relational instance:

Tuple#	<u>projectid</u>	<u>toolid</u>	<u>empid</u>
<u>1</u>	<u>P1</u>	<u>T1</u>	<u>E1</u>
<u>2</u>	<u>P2</u>	<u>T2</u>	<u>E2</u>
<u>3</u>	<u>P1</u>	<u>T2</u>	<u>E2</u>

Figure 6.4: A ternary relation with all independent attributes

The relation above does not have any FDs and MVDs since the attributes projectid, toolid and empid are independent; they are related to each other only by the pairings that have significant information in them.

For example, the first tuple of relation states that employee E1 works on P1 project, which uses tool T1. The key to the relation is the composite key (projectid, toolid, empid). The relation is in 4NF, but still suffers from the insertion, deletion, and update anomalies, as the information that Employee E1 can use tool T1 is redundant in tuple 3. Similarly, information like project P3 uses tool T1 cannot be inserted in the relation, as no employee has started working on the project. Likewise, on deleting tuple 2 from the relation, may delete the information like Employee E2 can use tool T2. Therefore, you need to decompose the relation to minimise the anomalies. As there are no FDs or MVDs in this relation, there is no basis of decomposition. You may try to see what happens when you decompose the relation into the following two relations:

<i>ProjectTool</i>	
<u>projectid</u>	<u>toolid</u>
P1	T1
P2	T2
P1	T2

<i>ProjectEmployee</i>	
<u>projectid</u>	<u>empid</u>
P1	E1
P2	E2
P1	E2

Figure 6.5: A decomposition of ternary relation of Figure 6.4

What happens when you take Join of the two relations on the attribute *projectid*:

Tuple#	<u>projectid</u>	<u>toolid</u>	<u>empid</u>
1	P1	T1	E1
2	P1	T1	E2
3	P2	T2	E2
4	P1	T2	E1
5	P1	T2	E2

Figure 6.6: ProjectTool JOIN ProjectEmployee

You may notice that the tuples 2 and tuple 4 in the relation state in Figure 6.6 were not existent in original relational state given in Figure 6.4. Thus, this decomposition is a lossy decomposition. So, what should be done to remove the anomalies?

In such situation, you may have to decompose the relation into three relations. Two of which are already shown in Figure 6.5. A third relation as shown in Figure 6.7 must be added.

<i>ToolEmployee</i>	
<u>toolid</u>	<u>empid</u>
T1	E1
T2	E2

Figure 6.7: The third relation

In order to obtain the original relation, you need to perform the following join operation:

ProjectTool JOIN ProjectEmployee JOIN ToolEmployee

Figure 6.6 represents (*ProjectTool JOIN ProjectEmployee*) which can be joined with *ToolEmployee*

Tuple#	<u>projectid</u>	<u>toolid</u>	<u>empid</u>
1	P1	T1	E1
2	P1	T1	E2
2	P2	T2	E2

<u>4</u>	<u>P1</u>	<u>T2</u>	<u>E1</u>
<u>3</u>	<u>P1</u>	<u>T2</u>	<u>E2</u>

Figure 6.8: *ProjectTool JOIN ProjectEmployee JOIN ToolEmployee*

Please note that Tuple 2 and Tuple 4 of Figure 6.6 will not be part of Figure 6.8, as there are no matching tuples in Figure 6.7. Thus, you can observe that Figure 6.4 and Figure 6.8 are identical.

Therefore, the relation ProjToolEmp (projectid, toolid, empid), which has no FD and MVD can be decomposed into three relations, which can be joined to form the original relation. This forms the concept of join dependency, which is explained next.

Join Dependency (JD): A relation R satisfies join dependency over the projections ($P_1, P_2, \dots, P_n$) if and only if every instance of R, the join of $P_1, P_2, \dots, P_n$ creates the instance of R.

For example, the instance of ternary relation ProjToolEmp (projectid, toolid, empid), as shown in Figure 6.4, when decomposed to two relations, as shown in Figure 6.5, does not satisfy the join dependency. This is shown in Figure 6.6. On addition of the third relation, as shown in Figure 6.7, the three projections ProjectTool (projectid, toolid), ProjectEmployee(projectid, empid) and ToolEmployee (toolid, empid) satisfies the join dependency as JOIN on the three projections, viz. ProjectTool (projectid, toolid), ProjectEmployee(projectid, empid) and ToolEmployee (toolid, empid), is the same as the instance of the relation. Thus, the three projections, as stated above, satisfies the join dependency.

6.4 FIFTH NORMAL FORM

The fifth normal form (5NF) (Project-Join normal form (PJNF)) deals with join-dependencies, which is a generalisation of the MVD. The aim of the fifth normal form is to have relations that cannot be decomposed further. A relation in 5NF cannot be constructed from several smaller relations.

A relation R is in 5NF if for all the join dependencies any one of the following clauses holds:

- (a) *join-dependency ($P_1, P_2, \dots, P_n$) is trivial (that is, one of the P_i 's is R)*
- (b) *Every P_i is a super key of R.*

For example, the instance of ternary relation ProjToolEmp (projectid, toolid, empid), as shown in Figure 6.4, has a Join Dependency (ProjectTool (projectid, toolid), ProjectEmployee(projectid, empid), ToolEmployee (toolid, empid)). Therefore, this relation is not in 5NF, as it violates the clauses of the 5NF, given above. You can decompose the relation ProjToolEmp (projectid, toolid, empid) into its projections as follows:

ProjectTool (projectid, toolid)

ProjectEmployee(projectid, empid)

ToolEmployee(toolid, empid)

Each of these three relations are in 5NF, as they have trivial join dependency. The instance of each of the decomposed relations is shown in Figure 6.5 and Figure 6.7. You may also observe that this decomposition is lossless join decomposition. It may be noted that every MVD is also a join dependency. Therefore, every PJNF (5NF) schema is also in 4NF. A relation schema that has Join Dependencies and suffers from anomalies, can be decomposed into projections, as per the join dependency. The new relations would be in PJNF schema.

☞ Check Your Progress 2

1) What is join dependency?

.....

.....

2) Define 5NF.

.....

.....

.....

3) Convert the relational instance of SUPPLY relation, as given in question 3 of check your progress 1. to 5NF.

.....

.....

.....

6.5 OTHER NORMAL FORMS

The researchers of database systems have found additional dependencies and normal forms. This section introduces the basic concept behind these forms. First, we define some additional types of dependencies.

Inclusion Dependency

The inclusion dependency has been designed for two specific types of database constraints that are not defined by the concept of FD, MVD and Join dependency. These two constraints are:

- Foreign key constraints
- Class/subclass relationships

Please note that these constraints are between two relations. Therefore, it requires a new form of formal definition. We define it in the context of foreign key constraints.

Consider two relations R1 and R2, which are related through a foreign key constraint such that a set of attributes in R1, say X, is the foreign key that references the relation R2 on a set of attributes, say Y. Please note that attribute sets X and Y must have similar number of attributes and similar domains, so that foreign key constraint is applicable. The inclusion dependency for such a situation will be defined as follows:

An **inclusion dependency** (denoted as $R1.X < R2.Y$) if the following relationship between the projections holds:

$$\pi_X(r1) \subseteq \pi_Y(r2) \quad (1)$$

Where r1 and r2 are the instances of relation R1 and R2 respectively at the same instance of time.

For example, consider the following two relational instances:

Relation: DEPT

deptID	deptName	deptlocation
D01	Marketing	Delhi
D02	Production	Mumbai
D03	HR	Delhi

Relation: EMP

empID	empName	salary	department
E01	ABC	30000	D01
E02	BCD	40000	D01
E03	CDE	45000	D02
E04	EFG	35000	D02

There exists a foreign key between the two relations, viz. department in EMP relation references deptID in DEPT relation. Therefore, the inclusion dependency $EMP.department < DEPT.deptID$ must hold for the given relational instances.

The equation (1) for this may be (assuming that relational instance of DEPT is dept and EMP is emp):

$\pi_X(r1)$ is $\pi_{department}(emp)$, which would be:

emp
department
D01
D02

and $\pi_Y(r2)$ is $\pi_{deptID}(dept)$, which would be:

dept
deptID
D01
D02
D03

You may observe that the inclusion dependency $EMP.department < DEPT.deptID$ holds, as

$\pi_{department}(emp) \subseteq \pi_{deptID}(dept)$.

Template Dependency

The template dependency is specified in the form of a template and can be used to represent any generic dependency. These dependencies can define any constraints in the form of a template. A template consists of two parts – the first part which shows the tuples that may exist in a relation is called the hypotheses, which are followed by the conclusion of the template.

A template dependency can be used to represent any dependency in this form. The following example shows how a FD can be represented using a template dependency.

Example: Consider a relation RESULT (studentid, courseid, grade) with the FD

$studentid, courseid \rightarrow grade$

Represent this FD using template dependency.

Description	<u>studentid</u>	<u>courseid</u>	grade
Hypothesis	S1	C1	G1

	S1	C1	G2
Conclusion	G1 = G2		

The following example shows, how MVDs can be represented using template dependency.

Example: Consider a relation EMPLOYEE (e#, project, tool) with the set of MVDs

$e\# \twoheadrightarrow project$ and $e\# \twoheadrightarrow tool$.

The following template dependency defines these MVDs:

Description	<u>e#</u>	<u>project</u>	<u>tool</u>
Hypothesis	E1	P1	T1
	E1	P2	T2
Conclusion	E1	P1	T2
	E1	P2	T1

One example of this MVD is shown with attribute values in the following table:

Description	<u>e#</u>	<u>project</u>	<u>tool</u>
Hypothesis	E001	DBMS	Python
	E001	Ecommerce	Virtualization
Conclusion	E001	DBMS	Virtualization
	E001	Ecommerce	Python

However, the actual use of template dependency may be to represent the constraints that cannot be represented using FD, MVD and join dependency. The following example explains this.

Example: Consider the relation EMP (empID, empName, salary, headID). In this relation the attribute headID represents empID of the head of the employee. You want to put a constraint in the relation that the employee cannot be given more salary than his/her head. The following template dependency would define this constraint:

Description	<u>empID</u>	<u>empName</u>	salary	headID
Hypothesis	E1	N1	S1	E2
	E2	N2	S2	E3
Conclusion	S2 >= S1			

The Domain-Key Normal Form (DKNF)

The Domain-Key Normal Form (DKNF) is beyond 5NF and proposes to make a set of relations free of all anomalies. The DKNF was defined as a consequence of Fagin's theorem that states:

"A relation is in DKNF if every constraint on the relation is a logical consequence of the definitions of keys and domains."

Let us define the key terms used in the definition above – *constraint*, *key* and *domain* in more detail. These terms were defined as follows:

Keys can be either the primary keys or the candidate key. Let R be a relation schema with $CK \subseteq R$. A key CK requires that CK be a super key for schema R such that $CK \rightarrow R$. Please note that a key declaration is a functional dependency but not all functional dependencies are key declarations.

Domain is the set of definitions of the contents of attributes and any limitations on the kind of data to be stored in the attribute. Let A be an attribute and **dom** be a set of values. The domain declaration can be stated as $A \subseteq \text{dom}$. It requires that the values of A in all the tuples of R be values from the set **dom**.

Constraint is a well-defined rule that is to be upheld by any set of legal data of R . A *general constraint* is defined as a predicate on the set of all the relations of a given schema. The MVDs, JDs are the examples of general constraints. However, a general constraint need not be just functional, multivalued, or join dependency. For example, the first two digits of your enrolment number represent the year in which you have taken admission. Assuming that MCA is the only programme of the university, whose maximum duration is 4 years. Also, assume that admission to this programme was started in 2021. Therefore, in the year 2026, there would be two types of students, viz. students whose enrolment number starts with 21 and students whose registration number starts with 22, 23, 24, 25 and 26. The registration of the students, whose registration starts with 21 would become invalid. Therefore, the general constraint for such a case may be: “If the first two digits of $t[\text{enrolmentnumber}]$ is 21, then $t[\text{marks}]$ are valid.” The t represents a tuple and *enrolmentnumber* and *marks* are the attributes.

The constraint suggests that the database design is not in DKNF. To convert this design to DKNF design, you need two schemas as:

Validstudentschema = (enrolmentnumber, subject, marks)

Invalidstudentschema = (enrolmentnumber, subject, marks)

Please note that the schema of valid students requires that the enrolment number of the students begin with 22. The resulting design is in DKNF.

Although DKNF is an aim of a database designer, it may not be implemented in a practical design.

Check Your Progress 3

- 1) Define Inclusion Dependencies.

.....
.....
.....

2) Define the template dependency.

.....
.....

3) What is the key idea behind DKNF?

.....
.....
.....

6.6 SUMMARY

This unit explains the concept of multi-valued dependencies, which causes a relation to have data redundancy. This causes a relation to have data anomalies. MVD is a consequence of having a set of attributes in a relation that determines more than one multi-valued attribute, which are independent of each other. MVD forms the basis of decomposition of a relation into 4NF relations. Further, certain relations do not have any FDs and MVDs but have anomalies. Such relations, in general, consist of more than two independent attributes. These relations contain join dependency, that is the relation has several projections, which can be joined losslessly to produce original relation. The join dependency forms the basis for 5NF decomposition. The unit also discusses about the inclusion and template dependencies, which are designed to represent the constraints that cannot be assigned using FDs, MVDs and join dependencies. Finally, the unit introduces the concept of DKNF. You may refer to the further readings for more details on these topics.

6.7 SOLUTIONS/ANSWERS

Check Your Progress 1

- 1) An MVD is a constraint due to multi-valued attributes. A constraint is multi-valued if all the following conditions hold in a relation:
- The relational must have at least three attributes out of which one should be multi-valued attribute.
 - One of the attributes can multi-determine the other two attributes.
 - The other two attributes, as stated above should be independent of each other.

- 2) There are two MVDs that exist in the relation. These are:

$ENAME \twoheadrightarrow PNAME$ and $ENAME \twoheadrightarrow DNAME$

Due to these two MVDs the relation suffers from insertion, update and deletion anomalies. This relation can be decomposed into the following projections, which are in 4NF.

EMP_PROJECT

ENAME	PNAME
Rohan	Big Data
Rohan	Machine Learning

EMP_DNAME

ENAME	DNAME
Rohan	AI Unit
Rohan	Analytics

- 3) The relational instance of SUPPLY relation shows that all three attributes are independent of each other as any supplier can supply any part to any project. Thus, there is no FD or MVD in the relation. Therefore, the relation is already in 4NF.

Check Your Progress 2

- 1) A join dependency is defined for a relation R and its projections, say $R_1, R_2, \dots, R_n$, as follows:
The join of relations $R_1, R_2, \dots, R_n$ must be equal to the relation R .
- 2) A relation is said to be in 5NF if either it has trivial join dependencies, or every projection R_i of the relation R is a super key of R .
- 3) All the attributes of relation SUPPLY are independent of each other. Does the relation have join dependency (SNAME, PARTNAME), (PARTNAME, PROJNAME), (PROJNAME, SNAME)?

The three projections for the given instance of SUPPLY would be:

R1

SNAME	PARTNAME
XYZ Pvt Ltd	Bolt
XYZ Pvt Ltd	Nut
ABC Ltd	Bolt
Info Comm Ltd	Nut
ABC Ltd	Nail

R2

PARTNAME	PROJNAME
Bolt	Big Data

Nut	Machine Learning
Bolt	Machine Learning
Nut	AI
Nail	Big Data

R3

SNAME	PROJNAME
XYZ Pvt Ltd	Big Data
XYZ Pvt Ltd	Machine Learning
ABC Ltd	Machine Learning
Info Comm Ltd	AI
ABC Ltd	Big Data

The join of the first two projections (R1 and R2) on PARTNAME would be:

JoinR1R2

SNAME	PARTNAME	PROJNAME
XYZ Pvt Ltd	Bolt	Big Data
XYZ Pvt Ltd	Bolt	Machine Learning
XYZ Pvt Ltd	Nut	Machine Learning
XYZ Pvt Ltd	Nut	AI
ABC Ltd	Bolt	Big Data
ABC Ltd	Bolt	Machine Learning
Info Comm Ltd	Nut	Machine Learning
Info Comm Ltd	Nut	AI
ABC Ltd	Nail	Big Data

The join of relations JoinR1R2 with R3 on SNAME, PROJNAME would be:

SNAME	PARTNAME	PROJNAME
XYZ Pvt Ltd	Bolt	Big Data
XYZ Pvt Ltd	Bolt	Machine Learning
XYZ Pvt Ltd	Nut	Machine Learning
ABC Ltd	Bolt	Big Data
ABC Ltd	Bolt	Machine Learning
Info Comm Ltd	Nut	AI
ABC Ltd	Nail	Big Data

Please observe that the joined relation is same as the instance of relation SUPPLY. Therefore, the join dependency (SNAME, PARTNAME), (PARTNAME, PROJNAME), (PROJNAME, SNAME) holds over the relation SUPPLY. To convert SUPPLY relation to 5NF, you may decompose it into the three projections R1, R2 and R3 as shown above. Please notice that each of the relation R1, R2 and R3 now have trivial join dependency, therefore, are in 5NF.

Check Your Progress 3

- 1) An inclusion dependency defines the referential constraint on two attributes. An inclusion dependency holds if a projection on an attribute of a relation, which may be a foreign key, is a proper subset of projection of another attribute of another relation, where it is the primary key.
- 2) Template dependency is a generic representation of various dependencies. It consists of two parts – the hypothesis and conclusion.
- 3) DKNF is the “ultimate normal form”. It defines the terms keys, domains and constraints.

